



P01-04 · T7–T10 aura-ingest — Scan, Hash, Pair, EXIF & Idempotent Import



The single hardest requirement on this page: **importing the same folder twice must insert zero rows the second time**, even if the first import was killed halfway, even if half the files were renamed, even if the drive letter changed. Everything below is built around that sentence.

1. `crates/aura-ingest/src/contract/ingest.rs` — FROZEN

```
///! FROZEN CONTRACT. The ingest boundary. PHASE-02 consumes ImportedFile to
///! build the preview pyramid; PHASE-30 consumes ImportReport for delivery
///! provenance. Neither may reach inside the importer.

use std::path::PathBuf;

use aura_core::{ContentHash, FileId, ImportId, PhotoId, ProjectId};
use serde::{Deserialize, Serialize};
use time::OffsetDateTime;

/// What the photographer asked for. Built by the UI, validated once, then
```

```

/// handed to the importer. An ImportPlan is serialisable so
a killed import
/// can be resumed from the plan plus the catalog, with no UI
involvement.
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ImportPlan {
    pub import_id: ImportId,
    pub project_id: ProjectId,
    /// Folders to walk. Symlinks are never followed.
    pub roots: Vec<PathBuf>,
    /// Copy files into a managed folder, or leave them where
they are.
    pub mode: ImportMode,
    /// Extensions to consider, lower case without dots. Empty
y means the
    /// built-in list. Never a wildcard: unknown formats must
be an explicit
    /// decision, not an accident.
    pub extensions: Vec<String>,
    /// Extract embedded JPEG previews during import for inst
ant thumbnails.
    pub extract_embedded_previews: bool,
    /// Treat files whose mtime is within this window as stil
l being written.
    pub settle_window_ms: u64,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deseri
alize)]
#[serde(rename_all = "snake_case")]
pub enum ImportMode {
    /// Reference in place. The photographer's folder structu
re is untouched.
    Reference,
    /// Copy into the project folder, verify the copy by has
h, then reference.

```

```

        CopyVerified,
    }

    /// One physical file that made it into the catalog.
    #[derive(Debug, Clone, Serialize, Deserialize)]
    pub struct ImportedFile {
        pub file_id: FileId,
        pub photo_id: PhotoId,
        pub abs_path: PathBuf,
        pub kind: FileKind,
        pub role: FileRole,
        pub size_bytes: u64,
        pub content_hash: ContentHash,
        pub mtime: OffsetDateTime,
        /// True when this exact file was already in the catalog.
        pub was_already_present: bool,
    }

    #[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
    #[serde(rename_all = "snake_case")]
    pub enum FileKind { Raw, Jpeg, Heif, Tiff, Png, SidecarXmp, Video, Other }

    #[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
    #[serde(rename_all = "snake_case")]
    pub enum FileRole { Primary, CompanionJpeg, Sidecar, Derivative }

    /// The honest account of what happened. Every number here is shown to the
    /// photographer; none of them are estimates.
    #[derive(Debug, Clone, Default, Serialize, Deserialize)]
    pub struct ImportReport {
        pub files_discovered: u64,
    }

```

```

    pub files_imported: u64,
    pub files_already_present: u64,
    pub files_quarantined: u64,
    pub photos_created: u64,
    pub bytes_hashed: u64,
    pub duration_ms: u64,
    /// Grouped by error code so the UI can say "3 files unre-
    /// adable" rather
    /// than listing three identical dialogs.
    pub quarantine_by_code: Vec<(String, u64)>,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deseri-
alize)]
#[serde(rename_all = "snake_case")]
pub enum ImportState { Scanning, Importing, Paused, Complete
d, Failed, Cancelled }

```

2. Scan — `crates/aura-ingest/src/scan.rs`

```

use std::path::{Path, PathBuf};

use aura_core::{AuraResult, errors::io as ioerr};
use walkdir::WalkDir;

use crate::contract::ingest::{FileKind, ImportPlan};

/// Default extension map. Unknown extensions are ignored, ne-
/// ver guessed.
pub const RAW_EXT: &[&str] = &[
    "cr2", "cr3", "crw", "nef", "nrw", "arw", "srf", "sr2",
    "raf", "dng",
    "orf", "rw2", "pef", "srw", "x3f", "3fr", "fff", "iiq",
    "mrw", "erf",
];

```

```

pub const JPEG_EXT: &[&str] = &["jpg", "jpeg", "jpe"];
pub const HEIF_EXT: &[&str] = &["heic", "heif", "avif"];
pub const TIFF_EXT: &[&str] = &["tif", "tiff"];
pub const SIDECAR_EXT: &[&str] = &["xmp"];
pub const VIDEO_EXT: &[&str] = &["mp4", "mov", "m4v", "avi",
    "mts", "m2ts"];

#[must_use]
pub fn classify(ext_lower: &str) -> FileKind {
    if RAW_EXT.contains(&ext_lower) { FileKind::Raw }
    else if JPEG_EXT.contains(&ext_lower) { FileKind::Jpeg }
    else if HEIF_EXT.contains(&ext_lower) { FileKind::Heif }
    else if TIFF_EXT.contains(&ext_lower) { FileKind::Tiff }
    else if ext_lower == "png" { FileKind::Png }
    else if SIDECAR_EXT.contains(&ext_lower) { FileKind::Side
carXmp }
    else if VIDEO_EXT.contains(&ext_lower) { FileKind::Video
}
    else { FileKind::Other }
}

/// Directory names we never descend into. Skipping these is
the difference
/// between a 40-second scan and a 6-minute one on a photogra
pher's drive.
const SKIP_DIRS: &[&str] = &[
    ".git", "node_modules", "$RECYCLE.BIN", "System Volume In
formation",
    ".Trashes", ".Spotlight-V100", ".fsevents", "Lightroom C
atalog Previews.lrdata",
    "Lightroom Catalog Smart Previews.lrdata", ".aura-cache",
    "Capture One Catalog",
    "@eaDir", ".dtrash",
];

#[derive(Debug, Clone)]

```

```

pub struct ScannedFile {
    pub abs_path: PathBuf,
    pub rel_path: String,
    pub file_name: String,
    pub ext: String,
    pub kind: FileKind,
    pub size_bytes: u64,
    pub mtime_ms: i64,
}

#[derive(Debug, Default)]
pub struct ScanOutcome {
    pub files: Vec<ScannedFile>,
    /// Files we refuse to touch, with the code explaining why. They become
    /// quarantine rows, not silent omissions.
    pub rejected: Vec<(PathBuf, aura_core::AuraError)>,
    pub dirs_visited: u64,
    pub bytes_total: u64,
}

/// Walk a root deterministically. Sorted by file name at each level, so the
/// order of insertion into the catalog is identical on every platform and on
/// every rerun. Determinism level D2 depends on this.
pub fn scan_root(root: &Path, plan: &ImportPlan, now_ms: i64)
-> AuraResult<ScanOutcome> {
    if !root.exists() {
        return Err(ioerr::not_found(root));
    }
    let mut out = ScanOutcome::default();
    let allowed: Vec<String> = if plan.extensions.is_empty()
    {
        RAW_EXT.iter().chain(JPEG_EXT).chain(HEIF_EXT).chain
        (TIFF_EXT)

```

```

        .chain(SIDECAR_EXT).map(|s| (*s).to_string()).col
lect()
    } else {
        plan.extensions.clone()
    };

    let walker = WalkDir::new(root)
        .follow_links(false) // a symlink loop is not our problem to solve
        .sort_by_file_name() // DETERMINISM: stable traversal order
        .max_depth(32);

    for entry in walker {
        let entry = match entry {
            Ok(e) => e,
            Err(e) => {
                // A single unreadable directory must not abort a 4,000-file import.
                let path = e.path().unwrap_or(root).to_path_buf();

                let io = e.io_error().map_or_else(
                    || ioerr::unreadable_str(&path, "directory traversal error"),
                    |io| ioerr::from_io(io, &path),
                );
                out.rejected.push((path, io));
                continue;
            }
        };

        if entry.file_type().is_dir() {
            let name = entry.file_name().to_string_lossy().to_string();
            if SKIP_DIRS.iter().any(|s| s.eq_ignore_ascii_case(&name)) { continue; }

```

```

        out.dirs_visited += 1;
        continue;
    }
    if !entry.file_type().is_file() { continue; }    // sockets, fifos, devices

    let path = entry.path().to_path_buf();
    let file_name = entry.file_name().to_string_lossy().to_string();

    // macOS AppleDouble stubs and hidden junk.
    if file_name.starts_with("._") || file_name == ".DS_Store" { continue; }

    let ext = path.extension().map(|e| e.to_string_lossy().to_lowercase()).unwrap_or_default();
    if !allowed.iter().any(|a| a == &ext) { continue; }

    // Windows path limit. We prepend the extended prefix where we can, and
    // quarantine when even that is not enough.
    if cfg!(windows) && path.as_os_str().len() > 32_000 {
        out.rejected.push((path.clone(), ioerr::path_too_long(&path)));
        continue;
    }

    let meta = match entry.metadata() {
        Ok(m) => m,
        Err(e) => {
            let err = e.io_error().map_or_else(
                || ioerr::unreadable_str(&path, "metadata unavailable"),
                |io| ioerr::from_io(io, &path),
            );
            out.rejected.push((path, err));
        }
    }

```



```

        continue;
    }
};

if meta.len() == 0 {
    out.rejected.push((path.clone(), ioerr::zero_byte
(&path)));
    continue;
}

let mtime_ms = crate::util::mtime_ms(&meta);

// Still being written? Card offloads and tethered sh
oots do this.
// Skip now, pick it up on the next pass rather than
hashing a partial file.
if now_ms - mtime_ms < i64::try_from(plan.settle_wind
ow_ms).unwrap_or(2000) {
    continue;
}

out.bytes_total += meta.len();
out.files.push(ScannedFile {
    rel_path: crate::util::rel_path_forward_slash(roo
t, &path),
    file_name,
    kind: classify(&ext),
    ext,
    size_bytes: meta.len(),
    mtime_ms,
    abs_path: path,
});
}
Ok(out)
}

```

Why `sort_by_file_name` is load-bearing

`walkdir` without sorting returns directory entries in filesystem order, which differs between APFS, NTFS and ext4, and can even differ between two runs on the same disk after a defragmentation. If insertion order varies, `photo_id` assignment varies, and any downstream tie-break — "pick the first of two equally good frames" — produces a different album. This one line is what makes determinism level D2 achievable at all.

Unicode and case collisions

macOS stores filenames in NFD (e + combining accent), Windows and Linux typically NFC. The same file copied between systems appears under two different byte sequences. Normalise before comparing, never before storing:

```
// crates/aura-ingest/src/util.rs
/// Compare-form of a path: NFC-normalised, forward slashes,
/// case preserved.
/// Used for duplicate detection only. The catalog always stores
/// the exact
/// bytes the filesystem gave us, so we can always reopen the
/// file.
#[must_use]
pub fn compare_key(rel_path: &str) -> String {
    use unicode_normalization::UnicodeNormalization;
    rel_path.nfc().collect::<String>()
}

/// A case-insensitive filesystem can present IMG_1.CR2 and i
/// mg_1.cr2 as one
/// file. On a case-sensitive one they are two. We record bot
/// h and let content
/// hashing decide whether they are actually the same bytes.
#[must_use]
pub fn case_fold_key(rel_path: &str) -> String { compare_key
    (rel_path).to_lowercase() }
```

3. Hashing — crates/aura-ingest/src/hash.rs

```
use std::{fs::File, io::Read, path::Path};

use aura_core::{AuraResult, ContentHash, errors::io as ioerr};

/// 1 MiB buffer. Larger buffers stop helping once the disk is saturated and
/// start hurting when eight threads each hold one.
const BUF: usize = 1024 * 1024;

/// Full-content BLAKE3. This is the identity of a file in AU
RA.
///
/// Why not sample the first and last megabyte, which would be ~50x faster?
/// Because two RAW files from the same camera in the same second share their
/// header and can share their tail, and a false duplicate silently deletes a
/// photograph. Full hashing at ~1 GB/s on eight cores costs about 25 s per
/// 220 GB wedding, which is inside budget and buys certainty.
pub fn hash_file(path: &Path, expected_size: u64) -> AuraResult<ContentHash, u64> {
    let mut file = File::open(path).map_err(|e| ioerr::from_io(&e, path))?;
    let mut hasher = blake3::Hasher::new();
    let mut buf = vec![0u8; BUF];
    let mut read_total: u64 = 0;

    loop {
        let n = file.read(&mut buf).map_err(|e| ioerr::from_i
```

```

o(&e, path))?;
    if n == 0 { break; }
    let chunk = buf.get(..n).ok_or_else(|| ioerr::unreadable_str(path, "short buffer"))?;
    hasher.update(chunk);
    read_total += n as u64;
}

// The file grew or shrank while we were reading it: a threaded capture or
// an in-flight card copy. Retryable, not a corruption.
if read_total != expected_size {
    return Err(ioerr::changed_during_read(path, expected_size, read_total));
}

Ok((ContentHash::from_bytes(*hasher.finalize().as_bytes()), read_total))
}

/// Parallel hashing with bounded memory. Rayon's default pool is sized to the
/// core count, which is wrong for IO-bound work on a spinning disk and roughly
/// right on NVMe; we cap it explicitly from the resource budget instead.
pub fn hash_batch(files: &[crate::scan::ScannedFile], threads: usize)
    -> Vec<(usize, AuraResult<(ContentHash, u64)>>)
{
    use rayon::prelude::*;
    let pool = rayon::ThreadPoolBuilder::new().num_threads(threads.max(1)).build();
    let Ok(pool) = pool else {
        return files.iter().enumerate().map(|(i, f)| (i, hash_file(&f.abs_path, f.size_b

```

```

ytes))).collect();
};
pool.install(|| {
    files.par_iter().enumerate()
        .map(|(i, f)| (i, hash_file(&f.abs_path, f.size_b
ytes)))
        .collect::<Vec<_>>()
    })
}

```

Storage	Hash threads	Reason
NVMe SSD	min(cores, 8)	Queue depth helps; BLAKE3 becomes the limit
SATA SSD or USB 3 SSD	4	Saturates the bus without thrashing
Spinning disk or SD card reader	1	Parallel reads cause seek storms and are slower than serial
Network share	2, with a warning	Latency-bound; also a candidate for refusal

4. Pairing — `crates/aura-ingest/src/pair.rs`

```

use std::collections::BTreeMap;

use crate::contract::ingest::{FileKind, FileRole};
use crate::scan::ScannedFile;

/// A photograph and every file that represents it.
#[derive(Debug, Clone)]
pub struct PhotoGroup {
    pub primary: usize,
    pub companions: Vec<(usize, FileRole)>,
}

/// Group by directory plus stem. `IMG_0421.CR2`, `IMG_0421.J

```

```

PG` and
/// `IMG_0421.xmp` in one folder are one photograph. The same
stem in a
/// different folder is a different photograph, because photo
graphers use
/// folders to mean something.
///
/// RAW wins as primary. If there is no RAW, the largest imag
e wins. A sidecar
/// is never primary. Ties are broken by the sorted scan orde
r, never by
/// filesystem order.
#[must_use]
pub fn group(files: &[ScannedFile]) -> Vec<PhotoGroup> {
    let mut buckets: BTreeMap<(String, String), Vec<usize>> =
BTreeMap::new();
    for (idx, f) in files.iter().enumerate() {
        let dir = f.rel_path.rsplit_once('/').map_or(String::
new(), |(d, _)| d.to_string());
        let stem = f.file_name.rsplit_once('.').map_or(f.file
_name.clone(), |(s, _)| s.to_string());
        // Adobe writes IMG_0421.CR2.xmp as well as IMG_0421.
xmp; strip twice.
        let stem = stem.rsplit_once('.').map_or(stem.clone(),
|(s, _)| s.to_string());
        buckets.entry((dir, crate::util::case_fold_key(&ste
m))).or_default().push(idx);
    }

    let mut groups = Vec::with_capacity(buckets.len());
    for (_key, members) in buckets {
        let pick_primary = members.iter().copied()
            .filter(|i| files.get(*i).is_some_and(|f| f.kind
!= FileKind::SidecarXmp))
            .max_by_key(|i| {
                let f = files.get(*i);

```

```

        let raw_bonus = u64::from(f.is_some_and(|f|
f.kind == FileKind::Raw)) << 40;
        raw_bonus + f.map_or(0, |f| f.size_bytes)
    ));

    let Some(primary) = pick_primary else {
        // Sidecar with no image: keep it, attached to no
        thing, so a later
        // import of the RAW can adopt it. Never discard
        photographer data.
        continue;
    };

    let companions = members.iter().copied().filter(|i| *
i != primary)
        .filter_map(|i| files.get(i).map(|f| (i, match f.
kind {
            FileKind::SidecarXmp => FileRole::Sidecar,
            FileKind::Jpeg | FileKind::Heif => FileRole::
CompanionJpeg,
            _ => FileRole::Derivative,
        }))).collect();

    groups.push(PhotoGroup { primary, companions });
}
groups
}

```

The `<< 40` bonus is a deliberate trick worth a comment in review: it makes "is a RAW" dominate any plausible file size, so a 60 MB RAW beats a 90 MB TIFF derivative without a branchy comparator.

5. EXIF — `crates/aura-ingest/src/exif.rs`

```

use std::{fs::File, io::BufReader, path::Path};

use aura_core::AuraResult;
use time::{OffsetDateTime, PrimitiveDateTime};

#[derive(Debug, Clone, Default)]
pub struct ExifFacts {
    pub capture_time: Option<OffsetDateTime>,
    pub capture_time_source: &'static str,
    pub camera_make: Option<String>,
    pub camera_model: Option<String>,
    pub lens_model: Option<String>,
    pub iso: Option<i64>,
    pub aperture: Option<f64>,
    pub shutter_us: Option<i64>,
    pub focal_len_mm: Option<f64>,
    pub orientation: u8,
    pub width_px: Option<i64>,
    pub height_px: Option<i64>,
    pub gps: Option<(f64, f64)>,
    /// Every tag, verbatim, for the exif table.
    pub raw_tags: Vec<(String, String)>,
}

/// EXIF parsing on wedding cards is adversarial input: camer
as write malformed
/// IFDs, third-party apps write circular pointers, and a cor
rupt card produces
/// arbitrary bytes. Rules:
/// 1. Never read more than 2 MiB looking for metadata.
/// 2. A parse failure is Warning severity, never ItemFaile
d. A photo with no
/// EXIF is still a photo, and dropping it would be the
worse bug.
/// 3. Cap every string at 256 chars before it reaches the

```



```

catalog.
pub fn extract(path: &Path) -> AuraResult<ExifFacts> {
    let mut facts = ExifFacts { orientation: 1, capture_time_
source: "unknown", ..Default::default() };

    let file = match File::open(path) {
        Ok(f) => f,
        Err(e) => return Err(aura_core::errors::io::from_io(&
e, path)),
    };
    let mut reader = BufReader::with_capacity(64 * 1024, fil
e.take(2 * 1024 * 1024));

    let parsed = match exif::Reader::new().read_from_containe
r(&mut reader) {
        Ok(p) => p,
        Err(e) => {
            tracing::warn!(target: "ingest", ext = ?path.exte
nsion(),
                error = %e, "exif unreadable, continuing with
out metadata");
            return Ok(facts);
        }
    };

    for field in parsed.fields() {
        let tag = field.tag.to_string();
        let mut value = field.display_value().to_string();
        value.truncate(256);
        facts.raw_tags.push((tag, value));
    }
    facts.raw_tags.sort(); // DETERMINISM:
stable exif row order
    facts.raw_tags.dedup_by(|a, b| a.0 == b.0);

    facts.camera_make = get(&parsed, exif::Tag::Make);

```

```

facts.camera_model = get(&parsed, exif::Tag::Model);
facts.lens_model   = get(&parsed, exif::Tag::LensModel);
facts.iso          = get_u32(&parsed, exif::Tag::PhotographicSensitivity).map(i64::from);
facts.orientation  = get_u32(&parsed, exif::Tag::Orientation)
    .and_then(|v| u8::try_from(v).ok()).filter(|v| (1..=8).contains(v)).unwrap_or(1);

// Exposure time as an exact rational, converted to integer microseconds.
if let Some(r) = get_rational(&parsed, exif::Tag::ExposureTime) {
    if r.denom != 0 {
        let us = (f64::from(r.num) / f64::from(r.denom))
* 1_000_000.0;
        facts.shutter_us = Some(us.round() as i64);
    }
}

facts.aperture     = get_rational(&parsed, exif::Tag::FNumber).map(|r| ratio(r));
facts.focal_len_mm = get_rational(&parsed, exif::Tag::FocalLength).map(|r| ratio(r));

// Capture time, in order of trustworthiness. DateTimeOriginal is when the
// shutter fired; DateTime is when the file was last written, which a
// camera-body clock reset or an editing app will have changed.
for (tag, source) in [
    (exif::Tag::DateTimeOriginal, "exif_original"),
    (exif::Tag::DateTimeDigitized, "exif_digitized"),
] {
    if let Some(raw) = get(&parsed, tag) {
        if let Some(t) = parse_exif_datetime(&raw, get(&p

```

```

arsed, exif::Tag::OffsetTimeOriginal)) {
    facts.capture_time = Some(t);
    facts.capture_time_source = source;
    break;
}
}
}

// GPS only if both coordinates are present and inside va
lid ranges. A
// corrupt IFD frequently yields lat = 91.0, and a wrong
venue on a map is
// worse than no venue.
if let (Some(lat), Some(lon)) = (gps_coord(&parsed, tru
e), gps_coord(&parsed, false)) {
    if (-90.0..=90.0).contains(&lat) && (-180.0..=180.0).
contains(&lon) {
        facts.gps = Some((lat, lon));
    }
}

Ok(facts)
}

```

Camera-clock reality, and why `capture_time_source` exists

At a real wedding two photographers shoot with three bodies, and at least one body's clock is wrong — typically by exactly one hour after a daylight-saving change, occasionally by years after a battery swap. PHASE-26 aligns those clocks. It can only do so if phase 01 records **where each timestamp came from** rather than silently falling back to file mtime and pretending it is a capture time.

`capture_time_source` is four bytes that make a whole later phase possible.

When no timestamp is available at all, we store `NULL` and set the source to `unknown`. We do **not** invent a time from the filename, even though `IMG_0421` strongly implies ordering. Inventing data is the one thing an ingest engine must never do.

6. Idempotent import — crates/aura-ingest/src/import.rs

```
use aura_catalog::Catalog;
use aura_core::{AuraResult, ContentHash, FileId, PhotoId};

use crate::contract::ingest::*;

/// The import algorithm, stated plainly:
///
/// 1. Record the source root, keyed by (project, absolute path). Reuse the
///    existing root_id if the photographer re-imports the same folder.
/// 2. Scan deterministically. Reject unreadable and zero-byte files into
///    quarantine without stopping.
/// 3. For every scanned file, look up (root_id, rel_path). If a row exists and
///    its fast_key matches, mark it seen and skip: no hash, no read.
/// 4. Otherwise hash it. If (project, content_hash) already exists elsewhere,
///    this is a copy: attach a new file row to the existing photo.
/// 5. Group unseen files into photographs, choose the primary, extract EXIF.
/// 6. Insert in batches of 200 inside one IMMEDIATE transaction per batch.
///    Commit after every batch so a kill loses at most 200 files of work.
/// 7. Update import_run counters in the same transaction as the rows they
///    describe, so the progress number can never disagree with the catalog.
```

```

pub fn run(cat: &Catalog, plan: &ImportPlan, cancel: &CancelT
oken) -> AuraResult<ImportReport> {
    let mut report = ImportReport::default();
    let started = cat.clock().monotonic_ms();

    for root in &plan.roots {
        let root_id = cat.upsert_source_root(plan.project_id,
root)?;
        let scan = crate::scan::scan_root(root, plan, cat.clo
ck().now_utc().unix_timestamp() * 1000)?;
        report.files_discovered += scan.files.len() as u64;

        for (path, err) in &scan.rejected {
            cat.quarantine(plan.project_id, plan.import_id, p
ath, err)?;
            report.files_quarantined += 1;
        }

        // Step 3: cheap skip pass. On a re-import of an unch
anged 4,000-file
        // wedding this eliminates 100 % of the hashing work.
        let known = cat.fast_keys_for_root(root_id)?; // BT
reeMap<String, String>
        let mut todo = Vec::with_capacity(scan.files.len());
        for f in scan.files {
            let fast_key = format!("{}", f.size_bytes, f.m
time_ms);
            match known.get(&f.rel_path) {
                Some(existing) if *existing == fast_key => {
                    cat.touch_file_seen(root_id, &f.rel_pat
h)?;
                    report.files_already_present += 1;
                }
                _ => todo.push(f),
            }
        }
    }
}

```

```

// Step 4-6: hash, group, insert in batches.
let threads = crate::tuning::hash_threads_for(root);
for chunk in todo.chunks(200) {
    if cancel.is_cancelled() {
        cat.set_import_state(plan.import_id, ImportState::Cancelled)?;
        report.duration_ms = cat.clock().monotonic_ms() - started;
        return Ok(report);
    }

    let hashes = crate::hash::hash_batch(chunk, threads);

    let mut hashed: Vec<(&crate::scan::ScannedFile, ContentHash)> = Vec::new();
    for (i, result) in hashes {
        let Some(file) = chunk.get(i) else { continue };

        match result {
            Ok((h, bytes)) => { report.bytes_hashed += bytes; hashed.push((file, h)); },
            Err(e) if e.is_retryable() => {
                // A file still being written. Try once more, then quarantine.
                match crate::hash::hash_file(&file.abs_path, file.size_bytes) {
                    Ok((h, bytes)) => { report.bytes_hashed += bytes; hashed.push((file, h)); },
                    Err(e2) => {
                        cat.quarantine(plan.project_id, plan.import_id, &file.abs_path, &e2)?;
                        report.files_quarantined += 1;
                    }
                }
            }
        }
    }
}

```

```

    }
    Err(e) => {
        cat.quarantine(plan.project_id, plan.
import_id, &file.abs_path, &e)?;
        report.files_quarantined += 1;
    }
}

let groups = crate::pair::group(&hashed.iter().ma
p(|(f, _)| (*f).clone()).collect::<Vec<_>>());
let inserted = cat.insert_photo_batch(plan, root_
id, &hashed, &groups)?;
report.files_imported += inserted.files;
report.photos_created += inserted.photos;
report.files_already_present += inserted.duplicat
es;

cat.bump_import_counters(plan.import_id, &repor
t)?;
}

// Files in the catalog that this scan did not see: m
ark absent, do not
// delete. A photographer who unplugged a drive has n
ot deleted a wedding.
cat.mark_absent_unseen(root_id, plan.import_id)?;
}

report.duration_ms = cat.clock().monotonic_ms() - starte
d;
cat.set_import_state(plan.import_id, ImportState::Comple
ted)?;
Ok(report)
}

```

The insert, where idempotence actually lives

```
-- One statement per file, inside the batch transaction.
-- ON CONFLICT DO UPDATE, never DO NOTHING: re-import must re
fresh last_seen_at
-- and resurrect a file previously marked absent, without cre
ating a new row.
INSERT INTO photo_file (
  file_id, photo_id, project_id, root_id, rel_path, file_nam
e, ext, kind, role,
  size_bytes, content_hash, mtime, fast_key, first_seen_at, l
ast_seen_at, is_present
) VALUES (?1,?2,?3,?4,?5,?6,?7,?8,?9,?10,?11,?12,?13,?14,?14,
1)
ON CONFLICT (root_id, rel_path) DO UPDATE SET
  last_seen_at = excluded.last_seen_at,
  is_present   = 1,
  size_bytes   = excluded.size_bytes,
  mtime        = excluded.mtime,
  fast_key     = excluded.fast_key,
  content_hash = excluded.content_hash
RETURNING file_id;
```

The `RETURNING file_id` matters: on conflict it returns the **existing** `file_id`, so the caller can tell an insert from an update and count them honestly rather than reporting 4,000 imports on a re-run.

7. Quarantine, and what the photographer sees

A quarantined file is not a failure of the import; it is a fact about the card. The import completes, and the report says so in plain language:

Situation	Code	What the UI says
3 files unreadable	<code>AURA-IO-1006</code>	"3 files could not be read and were set aside. Check the card before formatting it."

Situation	Code	What the UI says
1 zero-byte file	AURA-IO-1010	"1 file is empty, which usually means the card transfer was interrupted."
Nested too deep on Windows	AURA-IO-1005	"1 file is nested too deeply for Windows to open. Move it closer to the drive root and import again."
Drive removed at 60 %	AURA-IO-1003	"The drive was disconnected at 2,400 of 4,000 photos. Reconnect it and press Resume."

Quarantine rows are unique per `(project, abs_path)` with an `attempts` counter, so retrying an import does not produce four identical rows for the same broken file.

8. Tests for T7–T10

```
// crates/aura-ingest/tests/idempotence.rs

#[test]
fn second_import_of_identical_folder_inserts_nothing() {
    let (cat, plan, _dir) = fixture_with(400);
    let first = run(&cat, &plan, &CancelToken::new()).expect(
        ("first");
    let digest_a = cat.catalog_digest().expect("digest");

    let second = run(&cat, &plan, &CancelToken::new()).expect(
        ("second");
    let digest_b = cat.catalog_digest().expect("digest");

    assert_eq!(second.files_imported, 0, "re-import must insert zero files");
    assert_eq!(second.photos_created, 0);
    assert_eq!(second.files_already_present, first.files_imported);
    assert_eq!(digest_a, digest_b, "catalog must be byte-identical after re-import");
    assert_eq!(second.bytes_hashed, 0, "fast_key must prevent all rehashing");
}
```

```

}

#[test]
fn renaming_a_file_does_not_duplicate_the_photo() {
    // Same bytes at a new path: a new file row attached to the SAME photo_id.
}

#[test]
fn raw_and_jpeg_pair_become_one_photo() {
    let files = scan_fixture(&["IMG_1.CR2", "IMG_1.JPG", "IMG_1.xmp"]);
    let groups = pair::group(&files);
    assert_eq!(groups.len(), 1);
    assert_eq!(files[groups[0].primary].ext, "cr2");
    assert_eq!(groups[0].companions.len(), 2);
}

#[test]
fn same_stem_in_different_folders_are_two_photos() {
    let files = scan_fixture(&["day1/IMG_1.CR2", "day2/IMG_1.CR2"]);
    assert_eq!(pair::group(&files).len(), 2);
}

#[test]
fn scan_order_is_stable_across_runs_and_platforms() {
    let a: Vec<_> = scan_names();
    let b: Vec<_> = scan_names();
    assert_eq!(a, b);
    assert_eq!(a, insta::assert_yaml_snapshot!(a)); // golden, committed
}

#[test]
fn nfd_and_nfc_filenames_are_recognised_as_one_file() {

```

```

    // Cafe with a combining accent vs precomposed. One phot
o, not two.
}

#[test]
fn file_growing_during_hash_is_retried_then_quarantined() {
    // Writer thread appends bytes mid-hash; expect AURA-IO-1
007 then success
    // on retry, and quarantine if it keeps growing.
}

#[test]
fn corrupt_exif_still_imports_the_photo() {
    let facts = exif::extract(&fixture("corrupt-ifd.cr2")).ex
pect("no hard error");
    assert!(facts.capture_time.is_none());
    assert_eq!(facts.capture_time_source, "unknown");
}

#[test]
fn out_of_range_gps_is_dropped_not_stored() {
    let facts = exif::extract(&fixture("gps-lat-91.jpg")).exp
ect("ok");
    assert!(facts.gps.is_none());
}

#[test]
fn zero_byte_and_unreadable_files_are_quarantined_and_import_
completes() {
    let report = import_fixture_with_poison();
    assert_eq!(report.files_quarantined, 2);
    assert!(report.files_imported > 0);
    assert_eq!(count_quarantine_rows(), 2);
    // Re-running does not create more rows, only bumps attem
pts.
}

```

```

#[test]
fn cancel_stops_within_500ms_and_keeps_completed_work() {
    let cancel = CancellationToken::new();
    let t = std::thread::spawn({ let c = cancel.clone(); move
|| { std::thread::sleep(ms(250)); c.cancel(); }});
    let report = run(&cat, &plan, &cancel).expect("cancelled
cleanly");
    t.join().expect("join");
    assert!(report.files_imported > 0);
    assert_eq!(cat.import_state(plan.import_id).expect("stat
e"), ImportState::Cancelled);
    cat.integrity_check().expect("catalog intact");
}

#[test]
fn symlink_loop_does_not_hang_the_scan() {
    // a/b -> a. Scan must terminate and import the real file
    s.
}

```

9. Acceptance criteria for T7–T10

- ☐ 4,000-file fixture imports; counts in `ImportReport` match `SELECT COUNT(*)` exactly
- ☐ Re-import inserts 0 files, 0 photos, hashes 0 bytes, and leaves the catalog digest unchanged
- ☐ RAW + JPEG + XMP triple becomes one `photo` with three `photo_file` rows and the RAW as primary
- ☐ Identical stems in different folders stay separate photos
- ☐ NFD and NFC variants of one name resolve to one file
- ☐ Scan order snapshot test passes on macOS, Windows and Linux with the same committed golden
- ☐ Corrupt-EXIF fixture imports with `capture_time = NULL` and source `unknown`

- ☐ Invalid GPS is dropped; valid GPS is stored in `photo_gps`
- ☐ Zero-byte, unreadable, over-long-path files are quarantined with their exact codes; import still completes
- ☐ Re-running an import bumps `attempts` on existing quarantine rows instead of inserting duplicates
- ☐ Growing file triggers `AURA-I0-1007`, retries once, then quarantines
- ☐ Cancel acknowledged within 500 ms; work already committed survives; state is `cancelled`
- ☐ Symlink loop terminates
- ☐ Peak RSS during the 4,000-file import stays under 1.2 GB; number pasted
- ☐ Sustained hash throughput at or above 400 MB/s on the reference SSD; number pasted
- ☐ No `HashMap`, no `unwrap`, zero `ALLOW-BANNED` markers in `aura-ingest`

10. Claude Code prompt for T7–T10

Act as `SRC` (Senior Rust, Core). Implement T7 through T10 of PHASE-01 from this page. Copy `contract/ingest.rs` **verbatim**. Then implement `scan.rs`, `util.rs`, `hash.rs`, `pair.rs`, `exif.rs`, `tuning.rs`, `import.rs`, and every test in section 8 — including the six that are sketched as comments; implement them fully.

Hard constraints: hashing is full-content BLAKE3, never sampled; scan order is sorted and deterministic; symlinks are never followed; a metadata failure never aborts a run; EXIF parse failure is a `Warning` and the photo still imports; capture time is never invented from a filename; batches are 200 files with one `IMMEDIATE` transaction each; counters are updated in the same transaction as the rows they count.

Evidence I expect pasted back: `cargo test -p aura-ingest` in full; the double-import test output showing `files_imported = 0` and `bytes_hashed = 0` on the second run with identical catalog digests; the scan-order snapshot; peak RSS and hash throughput for the 4,000-file fixture; and the quarantine table contents after the poisoned-fixture run. Stop after T10.